

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

from tensorflow.keras.utils import to_categorical
```

```
text = """Machine learning is a branch of artificial intelligence,
Machine learning helps computers learn patterns,
deep learning is a subset of Machine learning,
Artificial intelligence is transforming the world,
Data science uses machine learning techniques.
"""

#convert into lowercase

text = text.lower()
print(text)
```

```
machine learning is a branch of artificial intelligence,
machine learning helps computers learn patterns,
deep learning is a subset of machine learning,
artificial intelligence is transforming the world,
data science uses machine learning techniques.
```

```
#Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts([text])
total_words = len(tokenizer.word_index) + 1
print("Total words:",total_words)
print(tokenizer.word_index)
```

```
Total words: 22
{'learning': 1, 'machine': 2, 'is': 3, 'a': 4, 'of': 5, 'artificial': 6, 'int
```

```
#sequence generation
input_sequences = []
for line in text.split('\n'):
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1,len(token_list)):
        n_gram_sequence = token_list[:i+1]
        input_sequences.append(n_gram_sequence)

print(input_sequences[:5])
```

```
[[2, 1], [2, 1, 3], [2, 1, 3, 4], [2, 1, 3, 4, 8], [2, 1, 3, 4, 8, 5]]
```

```
#padding (in order to make all the sequence same length)
max_sequence_len = max([len(x) for x in input_sequences])
input_sequences = np.array(
    pad_sequences(
        input_sequences,maxlen= max_sequence_len,padding='pre'
    )
)

print(input_sequences.shape)
```

```
(29, 8)
```

```
x = input_sequences[:, :-1]
y = input_sequences[:, -1]
y = to_categorical(y,num_classes=total_words)

print(x.shape,y.shape)
```

```
(29, 7) (29, 22)
```

```
#Dense layer
model = Sequential()
model.add(
    Embedding(
        input_dim= total_words,
        output_dim=64,
        input_length = max_sequence_len-1
    )
)

model.add(
    LSTM(128)
)

model.add(
    Dense(total_words,activation='softmax')
)

model.summary()

#compile model
model.compile(
    loss='categorical_crossentropy',
    optimizer = 'adam',
    metrics=['accuracy']
)
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
lstm_2 (LSTM)	?	0 (unbuilt)
dense_2 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)**Trainable params:** 0 (0.00 B)**Non-trainable params:** 0 (0.00 B)

```
#train model
history = model.fit(
    x,
    y,
    epochs=100,
    verbose=1
)

#Accuracy graph
plt.plot(history.history['accuracy'])
plt.title("Model Accuracy")
plt.xlabel('Epoch')
plt.ylabel("Accuracy")

plt.show()
```



```
Epoch 1/100
1/1  3s 3s/step - accuracy: 0.0690 - loss: 3.0898
Epoch 2/100
1/1  0s 54ms/step - accuracy: 0.1724 - loss: 3.0817
Epoch 3/100
1/1  0s 54ms/step - accuracy: 0.1724 - loss: 3.0733
Epoch 4/100
1/1  0s 54ms/step - accuracy: 0.1724 - loss: 3.0643
Epoch 5/100
1/1  0s 55ms/step - accuracy: 0.1724 - loss: 3.0544
Epoch 6/100
1/1  0s 57ms/step - accuracy: 0.1724 - loss: 3.0433
Epoch 7/100
1/1  0s 54ms/step - accuracy: 0.1724 - loss: 3.0305
Epoch 8/100
1/1  0s 55ms/step - accuracy: 0.1724 - loss: 3.0156
Epoch 9/100
1/1  0s 53ms/step - accuracy: 0.1724 - loss: 2.9979
Epoch 10/100
1/1  0s 70ms/step - accuracy: 0.1724 - loss: 2.9769
Epoch 11/100
1/1  0s 54ms/step - accuracy: 0.1724 - loss: 2.9517
Epoch 12/100
1/1  0s 52ms/step - accuracy: 0.1724 - loss: 2.9218
Epoch 13/100
1/1  0s 54ms/step - accuracy: 0.1724 - loss: 2.8872
Epoch 14/100
1/1  0s 54ms/step - accuracy: 0.1724 - loss: 2.8490
Epoch 15/100
1/1  0s 63ms/step - accuracy: 0.1724 - loss: 2.8120
Epoch 16/100
1/1  0s 59ms/step - accuracy: 0.1724 - loss: 2.7872
Epoch 17/100
1/1  0s 55ms/step - accuracy: 0.1724 - loss: 2.7866
Epoch 18/100
1/1  0s 57ms/step - accuracy: 0.1724 - loss: 2.7927
Epoch 19/100
1/1  0s 56ms/step - accuracy: 0.1724 - loss: 2.7814
Epoch 20/100
1/1  0s 53ms/step - accuracy: 0.1724 - loss: 2.7572
Epoch 21/100
1/1  0s 59ms/step - accuracy: 0.1724 - loss: 2.7328
Epoch 22/100
1/1  0s 55ms/step - accuracy: 0.1724 - loss: 2.7147
Epoch 23/100
1/1  0s 54ms/step - accuracy: 0.1724 - loss: 2.7032
Epoch 24/100
1/1  0s 56ms/step - accuracy: 0.2069 - loss: 2.6953
Epoch 25/100
1/1  0s 55ms/step - accuracy: 0.2759 - loss: 2.6879
Epoch 26/100
1/1  0s 54ms/step - accuracy: 0.2414 - loss: 2.6790
Epoch 27/100
1/1  0s 72ms/step - accuracy: 0.1724 - loss: 2.6674
Epoch 28/100
1/1  0s 55ms/step - accuracy: 0.1724 - loss: 2.6527
Epoch 29/100
1/1  0s 60ms/step - accuracy: 0.1724 - loss: 2.6349
Epoch 30/100
1/1  0s 54ms/step - accuracy: 0.1724 - loss: 2.6146
```

```
Epoch 31/100
1/1  0s 55ms/step - accuracy: 0.2414 - loss: 2.5925
Epoch 32/100
1/1  0s 58ms/step - accuracy: 0.2759 - loss: 2.5695
Epoch 33/100
1/1  0s 55ms/step - accuracy: 0.2069 - loss: 2.5463
Epoch 34/100
1/1  0s 55ms/step - accuracy: 0.1724 - loss: 2.5228
Epoch 35/100
1/1  0s 54ms/step - accuracy: 0.1724 - loss: 2.4980
Epoch 36/100
1/1  0s 55ms/step - accuracy: 0.2069 - loss: 2.4702
Epoch 37/100
1/1  0s 56ms/step - accuracy: 0.2069 - loss: 2.4384
Epoch 38/100
1/1  0s 55ms/step - accuracy: 0.2069 - loss: 2.4031
Epoch 39/100
1/1  0s 55ms/step - accuracy: 0.2414 - loss: 2.3655
Epoch 40/100
1/1  0s 54ms/step - accuracy: 0.2414 - loss: 2.3261
Epoch 41/100
1/1  0s 57ms/step - accuracy: 0.3103 - loss: 2.2844
Epoch 42/100
1/1  0s 59ms/step - accuracy: 0.3103 - loss: 2.2388
Epoch 43/100
1/1  0s 54ms/step - accuracy: 0.3448 - loss: 2.1882
Epoch 44/100
1/1  0s 64ms/step - accuracy: 0.3448 - loss: 2.1322
Epoch 45/100
1/1  0s 66ms/step - accuracy: 0.3103 - loss: 2.0732
Epoch 46/100
1/1  0s 56ms/step - accuracy: 0.3103 - loss: 2.0150
Epoch 47/100
1/1  0s 54ms/step - accuracy: 0.3448 - loss: 1.9549
Epoch 48/100
1/1  0s 52ms/step - accuracy: 0.3793 - loss: 1.8885
Epoch 49/100
1/1  0s 53ms/step - accuracy: 0.4138 - loss: 1.8231
Epoch 50/100
1/1  0s 55ms/step - accuracy: 0.4483 - loss: 1.7599
Epoch 51/100
1/1  0s 56ms/step - accuracy: 0.4483 - loss: 1.6886
Epoch 52/100
1/1  0s 55ms/step - accuracy: 0.4483 - loss: 1.6205
Epoch 53/100
1/1  0s 53ms/step - accuracy: 0.4483 - loss: 1.5504
Epoch 54/100
1/1  0s 56ms/step - accuracy: 0.5517 - loss: 1.4770
Epoch 55/100
1/1  0s 56ms/step - accuracy: 0.5862 - loss: 1.4079
Epoch 56/100
1/1  0s 54ms/step - accuracy: 0.6552 - loss: 1.3343
Epoch 57/100
1/1  0s 53ms/step - accuracy: 0.7241 - loss: 1.2664
Epoch 58/100
1/1  0s 54ms/step - accuracy: 0.8276 - loss: 1.1955
Epoch 59/100
1/1  0s 58ms/step - accuracy: 0.8276 - loss: 1.1325
Epoch 60/100
1/1  0s 54ms/step - accuracy: 0.8276 - loss: 1.0681
```

```

Epoch 61/100
1/1  0s 58ms/step - accuracy: 0.8276 - loss: 1.0124
Epoch 62/100
1/1  0s 67ms/step - accuracy: 0.8966 - loss: 0.9657
Epoch 63/100
1/1  0s 60ms/step - accuracy: 0.8276 - loss: 0.9524
Epoch 64/100
1/1  0s 54ms/step - accuracy: 0.8966 - loss: 0.8877
Epoch 65/100
1/1  0s 54ms/step - accuracy: 0.8966 - loss: 0.8308
Epoch 66/100
1/1  0s 56ms/step - accuracy: 0.8276 - loss: 0.8133
Epoch 67/100
1/1  0s 56ms/step - accuracy: 0.8966 - loss: 0.7483
Epoch 68/100
1/1  0s 58ms/step - accuracy: 0.9310 - loss: 0.7400
Epoch 69/100
1/1  0s 55ms/step - accuracy: 0.9310 - loss: 0.6808
Epoch 70/100
1/1  0s 56ms/step - accuracy: 0.8621 - loss: 0.6717
Epoch 71/100
1/1  0s 54ms/step - accuracy: 0.9310 - loss: 0.6235
Epoch 72/100
1/1  0s 59ms/step - accuracy: 0.9310 - loss: 0.6141
Epoch 73/100
1/1  0s 55ms/step - accuracy: 0.9655 - loss: 0.5732
Epoch 74/100
1/1  0s 54ms/step - accuracy: 0.9655 - loss: 0.5627
Epoch 75/100
1/1  0s 55ms/step - accuracy: 0.9655 - loss: 0.5289
Epoch 76/100
1/1  0s 53ms/step - accuracy: 0.9310 - loss: 0.5180
Epoch 77/100
1/1  0s 54ms/step - accuracy: 0.9310 - loss: 0.4893
Epoch 78/100
1/1  0s 54ms/step - accuracy: 0.9655 - loss: 0.4771
Epoch 79/100
1/1  0s 63ms/step - accuracy: 0.9655 - loss: 0.4549
Epoch 80/100
1/1  0s 65ms/step - accuracy: 0.9655 - loss: 0.4401
Epoch 81/100
1/1  0s 60ms/step - accuracy: 0.9655 - loss: 0.4237
Epoch 82/100
1/1  0s 54ms/step - accuracy: 0.9655 - loss: 0.4068
Epoch 83/100
1/1  0s 53ms/step - accuracy: 0.9655 - loss: 0.3948
Epoch 84/100
1/1  0s 56ms/step - accuracy: 0.9655 - loss: 0.3780
Epoch 85/100
1/1  0s 60ms/step - accuracy: 0.9655 - loss: 0.3681
Epoch 86/100
1/1  0s 60ms/step - accuracy: 0.9655 - loss: 0.3525
Epoch 87/100
1/1  0s 56ms/step - accuracy: 0.9655 - loss: 0.3439
Epoch 88/100
1/1  0s 56ms/step - accuracy: 0.9655 - loss: 0.3298
Epoch 89/100
1/1  0s 58ms/step - accuracy: 0.9655 - loss: 0.3215
Epoch 90/100
1/1  0s 58ms/step - accuracy: 0.9655 - loss: 0.3095
Epoch 91/100

```

```
#text generation function
def generate_text(seed_text,next_words):
    for _ in range(next_words):

        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences(
            [token_list],
            maxlen= max_sequence_len-1,
            padding='pre'
        )

        prediction = model.predict(token_list, verbose=0)
        predicted = np.argmax(prediction)

        output_word = ""
        for word,index in tokenizer.word_index.items():
            if index == predicted:
                output_word = word
                break

        print("\nPredicted word:",output_word)
        seed_text += " " + output_word

    return seed_text

#generate text
print(
    generate_text(
        "machine learning helps",
        5
    )
)

print(
    generate_text(
        "deep learning is a",
        5
    )
)

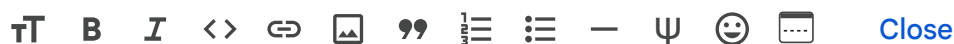
print(
    generate_text(
        "artificial intelligence is",
        5
    )
)
```

Predicted word: computers
machine learning helps computers

Predicted word: subset
deep learning is a subset

Predicted word: transforming

artificial intelligence is transforming

****What is RNN?****

Recurrent neural network is designed for influence future predictions.

****Why use RNN?****

As it remembers previous information and time-series data

****Difference between CNN and RNN?****

CNN has no memory and captures spacial features where RNN has memory and captures temporal previous inputs.

****What is Vanishing Gradient Problem?****

During backpropagation, gradients become the network to learn long-term dependencies

****Difference between RNN and LSTM?****

The main difference between LSTM and RNN learn from sequential data. LSTMs are more handling long-term dependencies, making them sequential data tasks.

What is RNN? Recurrent neural network is designed for sequential data where previous outputs influence future predictions.

Why use RNN? As it remembers previous information and is suitable for text, speech, and time-series data

Difference between CNN and RNN?

CNN has no memory and captures spacial features which process data independently where RNN has memory and captures temporal features and retain information from previous inputs.

What is Vanishing Gradient Problem?

During backpropagation, gradients become very small, making it difficult for the network to learn long-term dependencies.